GrOIL: Graph-Grounded Domain Ontology Induction with Constrained LLM Mediation

Maruf Ahmed Mridul
Rensselaer Polytechnic Institute
Troy, New York, USA
mridum@rpi.edu

Abid Talukder
Rensselaer Polytechnic Institute
Troy, New York, USA
taluka@rpi.edu

Oshani Seneviratne
Rensselaer Polytechnic Institute
Troy, New York, USA
senevo@rpi.edu

Abstract

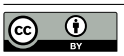
Constructing formal ontologies from domain documents requires simultaneously enforcing corpus grounding, vocabulary consistency, axiom-level expressivity, and end-to-end provenance, a combination no existing automatic system delivers. We present a seven-stage graph-grounded pipeline that converts domain documents into a complete, auditable Web Ontology Language (OWL) Terminological Box (TBox) without any unconstrained generation step. Documents are first encoded as Unified Discourse-Hypergraphs (UDH) capturing entity participation and discourse dependencies; subsequent stages transform this graph evidence into a class hierarchy, typed object and datatype properties, and restriction axioms, with Large Language Model (LLM) usage restricted to narrow, graph-grounded mediation tasks. A paired Assertional Box (ABox) population procedure grounds named individuals in the induced TBox, enabling SPARQL-based functional evaluation. Every emitted term carries a full decision chain from raw source passages through each pipeline stage, making the TBox directly auditable and suitable for targeted human refinement. Evaluated on the life insurance domain using two established benchmarks and a new 100-contract corpus spanning ten product types, our pipeline achieves strong results across all evaluation dimensions, outperforming direct and multi-agent LLM baselines on competency-question (CQ) coverage (**0.85** vs. 0.63 and 0.62 on one term-life contract; **0.77** vs. 0.40 and 0.44 on another contract), while also attaining high keyphrase coverage comparable to a manually-constructed reference ontology and strong performance on structured gap-and-overlap reasoning, all without any manual TBox engineering. Ontology growth analysis provides evidence consistent with vocabulary saturation at scale, demonstrating that the pipeline produces stable, reusable domain representations from large document corpora.

CCS Concepts

• **Computing methodologies** → **Knowledge representation and reasoning**; **Information extraction**; *Machine learning*; • **Information systems** → Information retrieval.

Keywords

Ontology Learning, Knowledge Representation, Closed-Vocabulary Prompting, Unified Discourse-Hypergraph, OWL TBox Induction, ABox Population, Provenance, LLM Hallucination Mitigation.



This work is licensed under a Creative Commons Attribution 4.0 International License.
CIKM '26, Rome, Italy
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2539-5/2026/11
https://doi.org/10.1145/3799682.3840910

ACM Reference Format:
Maruf Ahmed Mridul, Abid Talukder, and Oshani Seneviratne. 2026. GrOIL: Graph-Grounded Domain Ontology Induction with Constrained LLM Mediation. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26), November 07–11, 2026, Rome, Italy.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3799682.3840910

1 Introduction

Ontologies remain a central mechanism for turning domain knowledge into reusable, inspectable, and executable representations. In high-stakes domains such as insurance, legal compliance, healthcare, and finance, they provide more than semantic organization: they make policies, obligations, exclusions, eligibility conditions, and decision rules available for formal querying and downstream reasoning. However, constructing such ontologies from domain documents remains difficult. Expert-built ontologies are accurate but costly to produce and maintain, while automatic ontology learning systems have historically struggled to induce coherent and rich axiomatic structure [18, 30].

Automatic ontology construction methods, ranging from lexico-syntactic pattern mining and embedding-based clustering to recent LLM-based approaches, have significantly advanced the field, yet they share a persistent gap: systems either produce shallow outputs grounded in corpus evidence, or richer schema-level abstractions that lack grounding, consistency, and provenance [7, 8, 18, 30]. The root cause is architectural: existing pipelines either delegate too much to the LLM, inviting hallucination, redundancy, and ungrounded term invention, or too little, forfeiting the semantic judgment that only a language model can supply. No existing system jointly enforces corpus grounding, vocabulary-level control, axiom-level expressivity, and end-to-end provenance within the same pipeline.

We argue that ontology induction should be treated as a grounded decision pipeline in which the document corpus first supplies a structured evidence graph, and the LLM is invoked only for bounded semantic decisions that resist deterministic resolution. Rather than asking the model to generate an ontology freely, every LLM call is restricted to a local modeling decision over graph-derived evidence and a closed output vocabulary, enforced at generation time via closed-vocabulary prompting, covering tasks such as naming clustered candidates, resolving ambiguous domain and range, and placing classes in an underspecified hierarchy. This design makes the LLM's role precise and auditable: it fills bounded gaps in structured reasoning, never authors structure freely.

We present a seven-stage pipeline that realizes this design, converting documents into UDH using Talukder et al.'s [47] method and transforming the resulting graph evidence into a complete,

traceable OWL TBox. Every emitted term carries a complete decision chain, making the induced TBox auditable and suitable for targeted human refinement. We further extend the pipeline with an ABox population procedure enabling SPARQL-based functional evaluation over populated ontologies.

We evaluate the system in the life insurance domain using two benchmarks [33, 46], and a new 100-contract corpus spanning diverse product types. Insurance contracts are a difficult testbed: state-of-the-art models reading contracts directly still make systematic errors on coverage determination tasks, confirming that surface language understanding alone is insufficient and that structured, provenance-grounded representations carry genuine value [33]. Our induced ontology achieves high scores on this benchmark, matching the strongest LLM baseline, without any manual TBox engineering, while additionally providing deterministic reproducibility and structured evidence trails. The contributions of this work are:

- **End-to-end TBox and ABox induction.** A graph-grounded pipeline that induces a complete OWL TBox and populated ABoxes from domain documents, with every class, property, restriction, and individual carrying full provenance annotations traceable to source passages.
- **Constrained LLM mediation.** The pipeline assigns LLMs narrow, well-defined roles within a graph-grounded ontology induction process. LLMs only mediate localized decisions, such as cluster naming, domain and range resolution, and class placement, while closed-vocabulary prompting constrains term-selection steps.
- **An extended 100-contract life insurance dataset.** We expand the 10-contract benchmark of Mridul et al. [33] to 100 synthetic contracts across diverse product types, enabling scalability and ontology-growth analysis beyond small benchmarks.
- **Comprehensive ontology evaluation.** We evaluate structural quality, CQ coverage, keyphrase coverage, ontology growth, and executable gap and overlap analysis, showing strong functional performance without manual TBox engineering.

The generated resources and our source code are available at https://github.com/brains-group/GrOIL, with the accompanying README providing detailed instructions for reproducibility.

2 Related Work

Ontology engineering has long been recognized as a labor-intensive, expert-driven discipline, with a persistent *knowledge acquisition bottleneck* at every phase from initial modeling through long-term maintenance [22, 38, 45, 49]. The ontology learning field emerged to attack this bottleneck by automating extraction from text [16, 31]. Early work established lexico-syntactic patterns for taxonomy induction [24]: an approach that remains competitive on large corpora [43], and the *ontology learning layer cake* framing organized pipelines from term extraction through concept formation, relation extraction, and axiom induction [6, 13]. Systems such as OntoLearn demonstrated that domain corpora can yield term hierarchies without manual seeding [36, 50], yet scalable end-to-end solutions have remained elusive: relation extraction and axiom induction are consistently the hardest open problems, and recent reviews confirm that modern methods still assemble isolated components rather than delivering unified frameworks [18, 30].

Pre-trained language models transformed the upstream information extraction task that any ontology learning pipeline depends on [17], and dense sentence representations enabled embedding-based deduplication of surface-variant concepts at scale [42]: a technique our pipeline employs directly. LLMs have since attracted attention as ontology engineering assistants, but the landscape remains fragmented. LLMs4OL established that LLMs handle isolated sub-tasks such as term typing but degrade on more complex relational extraction tasks [7]; OLLM showed that end-to-end taxonomic backbone construction is feasible but that domain adaptation requires fine-tuning on domain-specific examples [29]. Lippolis et al. show that careful prompting can outperform novice modelers on small benchmarks, yet without systematic corpus grounding [27]; Saeedizade and Blomqvist enumerate OWL modeling options from requirements but delegate selection to humans [44]; Kommineni et al. pair open-source LLMs with an LLM judge but find quality varies sharply with model size [26]. Bakker et al. document persistent failures including hallucination, incomplete coverage, and unstable property typing [8], while Norouzi et al. demonstrate that ontology population is reliable when the LLM is given a closed vocabulary [37]—a principle we extend to domain and range assignment and restriction induction. Domain-oriented LLM efforts follow the same pattern: Chowdhury et al. and Abolhasani and Pan apply guided prompting to abstract cultural concepts and technical texts, respectively [1, 15], but both are corpus-specific. Talukder et al. directly precede our work with a multi-agent LLM baseline for insurance ontology generation that our pipeline substantially surpasses [46].

Enforcing valid ontology term output requires constrained decoding; structured generation has become standard for reliable LLM deployment in machine-facing pipelines [10, 20, 51]. At the broader LLM–KG integration level, Pan et al. map three integration regimes (KG-enhanced LLMs, LLM-augmented KGs, synergized systems) [41], and Zhu et al. empirically confirm that large models serve better as inference assistants than few-shot extractors [54]; Bian et al. survey the resulting schema-based versus schema-free construction paradigms [12]. Document-level relation extraction has benefited from graph-structured representations that capture multi-hop reasoning paths beyond sentence boundaries [52], motivating our graph formalism. Surface-variant deduplication via agglomerative clustering on dense embeddings [34] is a prerequisite for any induction system that must avoid TBox redundancy.

Domain ontologies for insurance and legal text confirm that formal ontologies enable executable SPARQL-based reasoning that LLM inference cannot replicate [2, 14, 19, 35], and restriction axioms such as SWRL rules have clear downstream utility in high-stakes settings [3, 25]. Competency questions and LLM-as-judge panels provide the standard evaluation frame [5, 11, 28, 53], complemented by keyphrase coverage [33], all of which we adopt. Across this body of work a consistent gap emerges: classical methods are corpus-grounded but shallow; LLM-based methods produce syntactically valid OWL but lack global consistency and provenance; domain-specific efforts require substantial manual TBox engineering. Our pipeline sits at the intersection: grounding every LLM call in corpus-derived graph evidence, enforcing outputs against a closed ontology vocabulary, and materializing end-to-end provenance annotations on every emitted term.

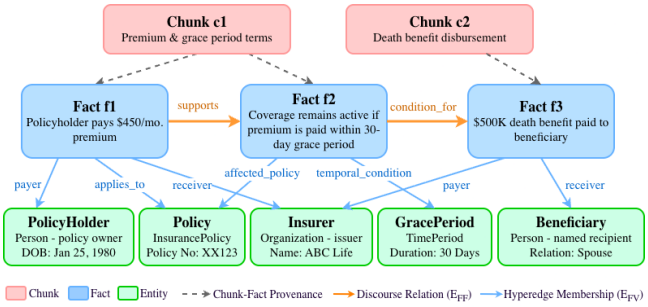


Figure 1: Input graph structure: chunk nodes (provenance anchors), fact nodes (hyperedges over entities and discourse graph nodes), and entity nodes with typed roles and relational anchors.

3 Methodology

Given a corpus $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$ of domain documents, the task is to produce a formal OWL TBox $\mathcal{T} = (\mathcal{C}, \mathcal{P}, \mathcal{A})$, where $\mathcal{C}$ is a set of named classes organized in an `rdfs:subClassOf` hierarchy, $\mathcal{P}$ is a set of object and datatype properties with explicitly typed domain and range, and $\mathcal{A}$ is a set of TBox restriction patterns (existential and universal restrictions, disjointness assertions, and cardinality constraints), all grounded in evidence drawn from $\mathcal{D}$.

3.1 Input Preparation

Each document $d \in \mathcal{D}$ is first converted into a structured graph representation before any ontological reasoning is performed. We adopt the UDH formalism [47], which encodes document knowledge along two complementary axes: *entity-centric participation*, capturing which entities co-occur within the same proposition and in what functional roles, and *narrative connectivity*, capturing the logical and conditional dependencies between propositions across the document.

Graph Structure: A UDH for a document is defined as $\mathcal{G} = (C, F, V, E_{CF}, E_{FV}, E_{FF})$, as illustrated in Figure 1. *Chunks* $C = \{c_1, \ldots, c_k\}$ are overlapping source-text windows used as provenance anchors. *Facts* $F = \{f_1, \ldots, f_m\}$ are atomic, self-contained propositions extracted from the text, each augmented with thematic topics, and a set of participating entities. Critically, facts occupy a dual structural role: they are simultaneously *hyperedges* over the entities that participate in them (via E_{FV}) and *nodes* in a directed discourse graph (via E_{FF}). *Entities* $V = \{v_1, \ldots, v_p\}$ are richly typed fingerprints, each carrying a canonical name, an ontological type (e.g., `PERSON`, `ORGANIZATION`), a contextual micro-role grounding the entity's function within its passage, and a set of discriminating features such as policy numbers or relational attributes.

The two edge sets encode the two structural axes described above. The hyperedge-entity incidence relation $E_{FV} \subseteq F \times V \times \mathcal{L}$, where $\mathcal{L}$ is a vocabulary of edge labels, records the typed functional role of each entity in a fact (e.g., *payer*, *receiver*, *affected_policy*). The discourse edge relation $E_{FF} \subseteq F \times F \times \mathcal{L} \times [0, 1]$ records directed, labeled narrative relationships between facts (e.g., *supports*, *condition_for*), together with a confidence score. Together, these two edge sets make UDH graphs a rich substrate for ontology induction as evidenced in Figure 1 : E_{FV} provides direct, corpus-grounded evidence for property axioms, while E_{FF} surfaces logical dependencies that would be invisible to entity or triple-based representations, a richness of structure that neither a flat chunk nor a binary triple can express.

Document-to-Graph Conversion: We apply the full UDH construction pipeline [47] to each document independently, producing a collection of per-document graphs $\{\mathcal{G}_d\}_{d \in \mathcal{D}}$, each serialized as a typed GraphML file in which node records carry the full entity fingerprint metadata and edge records carry role labels, discourse relation types, and confidence scores. No raw document text is accessed beyond this point; all subsequent stages operate exclusively on the structured graph representation.

Graph Cleaning: Before ontology construction begins, each graph $\mathcal{G}_d$ is passed through a two-step cleaning pipeline applied to all edges $E_{FV} \cup E_{FF}$. **Step 1** removes null-like edges (such as *none*, *unrelated*, or *independent*), discards low-confidence edges, and normalizes labels via a synonym map of a predefined set of high-frequency pre-defined canonical relation types. **Step 2** globally clusters surviving labels using agglomerative clustering [34] to resolve near-paraphrases not covered by the map, flagging suspicious nulls and rare labels for removal, and eliminating self-loops and duplicate triples. The cleaned graphs $\{\mathcal{G'}_d\}$ then serve as input to Stage 1 of the ontology construction pipeline.

3.2 Pipeline Overview

Ontology construction from $\{\mathcal{G}'_d\}_{d \in \mathcal{D}}$ proceeds in seven stages, summarized in Figure 2. The pipeline prioritizes graph-derived, corpus-grounded evidence before invoking LLMs for bounded decisions, including naming, cluster formalization, ambiguous domain and range assignment, and underspecified hierarchy construction. This restricts the model's freedom, grounds its outputs in corpus evidence, and reduces reliance on large proprietary models, making small open-source LLMs sufficient.

Stage 1: Parsing and Typed Pool Construction

Stage 1 reads the cleaned GraphML files in configurable batches and produces, for each document, a **typed pool**: the partitioned sets of chunk, fact, and entity records, together with an edge bucket index mapping each bucket label to its edges. Each entity record carries five fields: an identifier, a name, a normalized type, a roles list, and a set of relational anchors. The `roles` field deserves attention: entities frequently occupy multiple functional positions encoded as compound role strings (e.g., PERSON : *Policy Owner and Insured*). We split these along conjunctive delimiters, yielding atomic roles that each become independent class candidates in Stage 2, with co-occurring roles providing sibling signals for Stage 5.

The relational anchors (e.g., *DOB: January 25, 1980; Age: 44; Underwriting Class: Standard Plus Non-Tobacco*) are preserved verbatim as structured metadata, providing a pool of data property candidates recoverable in later stages.

Stage 2: Concept Candidate Extraction

Stage 2 aggregates the typed pools across all documents in $\mathcal{D}$ and extracts a flat **candidate pool** Γ. Each candidate $\gamma \in \Gamma$ is a tuple:

$$\gamma = (\underbrace{s}_{\text{string}}, \underbrace{o}_{\text{origin}}, \underbrace{p}_{\text{parent}}, \underbrace{f}_{\text{freq}}, \underbrace{df}_{\text{doc_freq}}, \underbrace{m}_{\text{meta}})$$

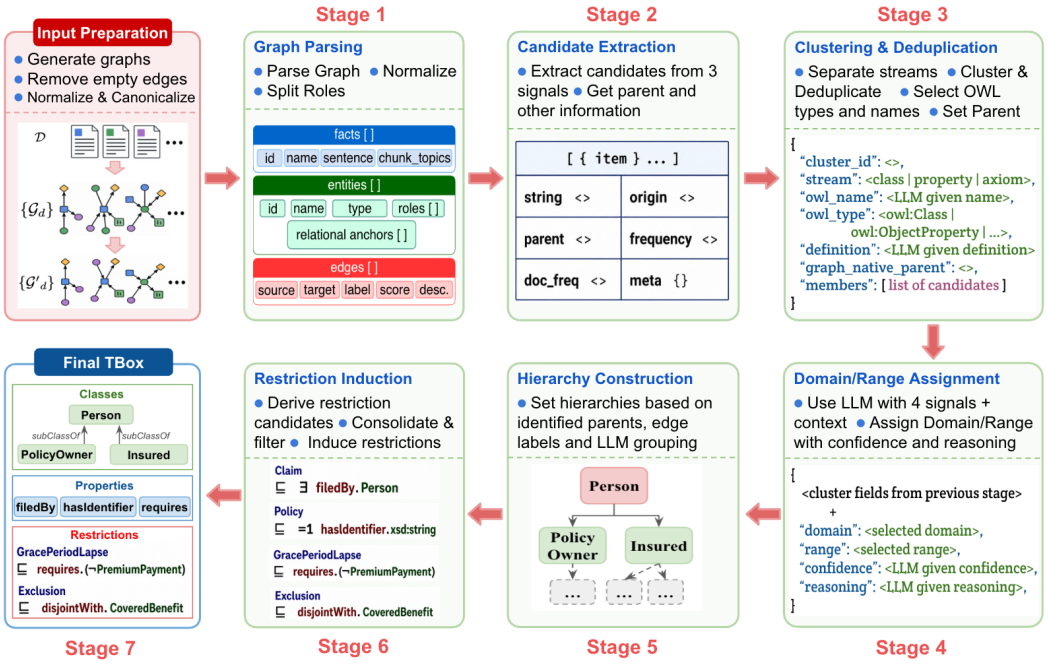


Figure 2: Overview of the proposed ontology construction pipeline.

where $o \in \{\text{ENTITY_TYPE}, \text{ENTITY_ROLE}, \text{EDGE_LABEL}, \text{FACT_NAME}, \text{CHUNK_TOPICS}\}$ records the graph signal from which the candidate was extracted, p encodes a structural subclass hypothesis for role-derived candidates (a role observed under type t is hypothesized to denote a subclass of the class corresponding to t), and m carries auxiliary evidence (primarily source and target type distributions for edge-label candidates). Candidates are extracted from the following signals.

Entity Types and Roles: For every entity in the corpus, one candidate is emitted for its normalized type and one for each atomic role in its roles list. The origin tag distinguishes these: ENTITY_TYPE for type-derived candidates and ENTITY_ROLE for role-derived ones. For the latter, the `parent` field records the entity type under which the role was observed, directly encoding the ontological hypothesis that the role denotes a subclass of the corresponding type class. An instance of *policy owner* observed under a PERSON entity encodes *PolicyOwner* $\sqsubseteq$ *Person* without consulting any external lexical resource. This structural parent link propagates to Stage 3, where it provides high-confidence subclass placement if observed consistently across the corpus.

Edge Labels with Type Context: For each edge in the corpus, its label is emitted as an EDGE_LABEL candidate. The metadata field records the empirical distributions of source and target entity types observed for that label across the corpus. A label such as *has death benefit option*, consistently observed between POLICY source nodes and fact-node targets, carries a distributional prior exploited for domain and range assignment in Stage 4. Collecting these distributions at extraction time keeps subsequent stages independent of the raw graph files.

Fact Names: Fact node names are emitted as FACT_NAME candidates, surfacing propositional predicates (such as *policy lapse after grace period* or *premium payment frequency*) that are unreachable through entity or edge signals alone. Fact names also feed the axiom stream in Stage 3, where names containing modal or conditional language are treated as axiom candidates rather than plain property candidates.

Chunk Topics: Thematic topic strings associated with facts are emitted as CHUNK_TOPICS candidates. These are extracted from the `chunk_topics` field stored on fact nodes, since Stage 1 records topics via facts rather than directly via chunk records, and serve as supplementary class candidates in Stage 3's class stream.

After candidate extraction, frequency filtering is applied per origin to ensure candidates are representative across the corpus rather than inflated by a single verbose document, controlling for distributional skew and improving generalizability.

Stage 3: Deduplication and Semantic Clustering

The candidate pool Γ contains considerable lexical redundancy: surface variants of the same underlying concept must be consolidated before OWL terms can be introduced. Stage 3 performs this via embedding-based agglomerative clustering followed by LLM-assisted normalization, operating independently across three candidate streams.

Stream Separation: Candidates are partitioned into three streams. The **class stream** takes candidates of origin ENTITY_TYPE, ENTITY_ROLE, and CHUNK_TOPICS. The **property stream** takes EDGE_LABEL candidates and FACT_NAME candidates whose string does not match a restriction keyword pattern. The **restriction stream** takes FACT_NAME candidates whose underlying proposition requires a logical operator to express in OWL. Since surface wording alone is unreliable for this distinction, stream assignment for FACT_NAME candidates is delegated to an LLM, which classifies

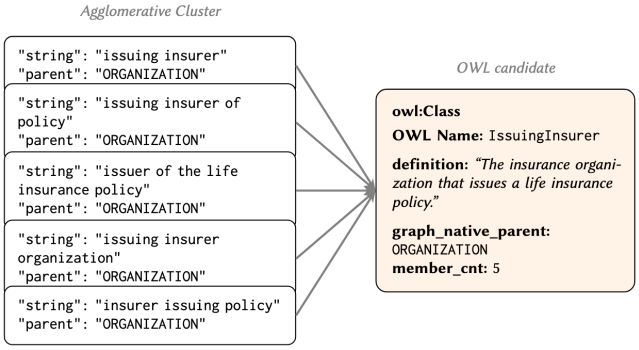


Figure 3: Example clustering outcome for a class-stream cluster. Five surface variants, all observed under entity type ORGANIZATION, are merged by agglomerative clustering into a single `owl:Class` candidate. The graph-native parent is assigned here, because all variants carry the same parent information from the graph.

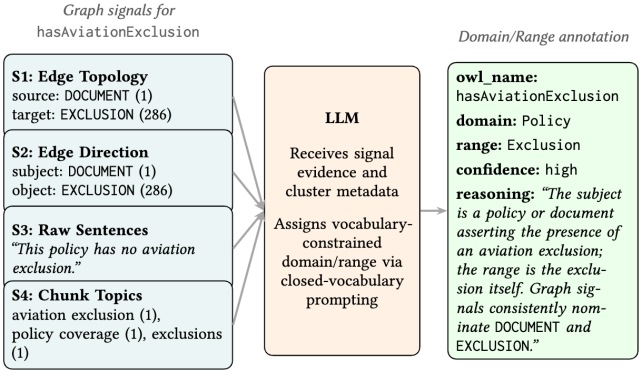


Figure 4: Domain and range assignment for `hasAviationExclusion`. Four graph signals guide the decision: S1 and S2 nominate `DOCUMENT` as domain and `EXCLUSION` as range, while S3 and S4 provide supporting natural-language context. The LLM synthesizes these signals under closed-vocabulary prompting, producing a high-confidence assignment with an explanatory reasoning.

each using its source sentence as evidence. Streams are processed independently to prevent cross-stream conflation: a string that functions as both a class name and a relation predicate in different contexts is handled separately in each stream.

Embedding-Based Clustering: Within each stream, candidate strings are encoded using `all-mpnet-base-v2`[1] with origin-aware prompt templates that inject domain context before embedding. An ENTITY_ROLE candidate with parent p is rendered as *"insurance role of p: {string}"*, an EDGE_LABEL candidate as *"relationship in insurance policy: {string}"*, and so on. The resulting normalized embeddings are clustered with **agglomerative clustering** [34] (average linkage, cosine distance, similarity threshold). The threshold-based stopping criterion means no pre-specified k is required; the number of clusters is determined entirely by the data geometry. Average-linkage cosine clustering is well-suited here because, after frequency filtering, the candidate strings within each stream are already semantically coherent: near-paraphrase variants (see Figure 3) lie within a tight cosine neighborhood and merge naturally, while semantically distinct concepts remain separated.

LLM-Assisted Normalization: For each cluster, the top-k members by frequency are presented to the LLM, which assigns a canonical OWL name in PascalCase (for classes) or camelCase (for properties) and generates a concise natural-language definition. The LLM also assigns the type (`owl:ObjectProperty` or `owl:DatatypeProperty`) for properties and `RestrictionPattern` for the restrictions at this stage. After naming, parent assignment for class-stream clusters proceeds by structural majority vote. For any cluster that contains at least one ENTITY_ROLE member, the `parent` field of each such member (set in Stage 2, Section 3.2) is collected, and the most frequent parent type is recorded as `graph_native_parent` at confidence level *high*. For clusters with no role members, or where role members lack consistent parent information, the parent field is left unresolved here and delegated to Stage 5.

[1]https://huggingface.co/sentence-transformers/all-mpnet-base-v2

Stage 4: Domain and Range Assignment

This stage annotates each `owl:ObjectProperty` and `owl:DatatypeProperty` cluster from the property and restriction streams with a typed domain and range. The key design choice is to treat this as a multi-signal decision problem: four complementary graph signals are computed from the corpus and presented collectively to an LLM, which synthesizes them into a final assignment under a hard vocabulary constraint enforced at generation time.

S1 (edge topology) aggregates the empirical source and target node-type distributions over all edges whose label is non-null and loosely matches any member of the property cluster. The mode source type is reported as a domain candidate and the mode target type as a range candidate. A minimum edge count threshold filters out property clusters with insufficient corpus evidence before any nomination is made.

S2 (edge direction) specifically examines the ENTITY_FACT and FACT_ENTITY edge buckets. An entity appearing as the source of an ENTITY_FACT edge with a matching label is a domain candidate; an entity appearing as the target of a FACT_ENTITY edge is a range candidate. This signal attends to the directed argument structure encoded in the graph topology rather than the global type distribution.

S3 (raw sentences) and S4 **(chunk topics)** supply natural-language evidence directly to the LLM: S3 passes up to k_s fact-node sentences verbatim, while S4 passes the top k_t thematic topic strings ranked by corpus frequency. Both are used as-is without further parsing.

LLM Assignment with closed-vocabulary prompting: All four signals, together with the cluster's OWL name, definition, and top member strings, are presented to the LLM in a structured prompt. The LLM selects domain and range from a closed vocabulary: the set of OWL class names produced in Stage 3 for object properties, and a fixed set of XSD datatypes for datatype properties.

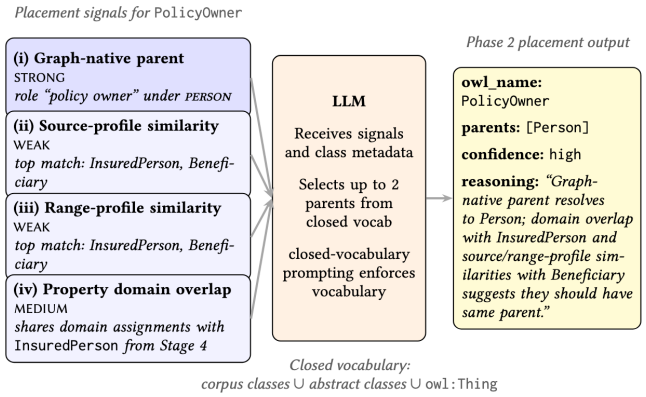


Figure 5: Hierarchy construction for `PolicyOwner`. Signal (i) gives a strong graph-native parent from Stage 3 metadata; signals (ii)–(iii) provide structural evidence from source and range edge-type profile similarity; and signal (iv) identifies sibling relations through Stage 4 property-domain overlap. The LLM selects at most two parents from a closed vocabulary enforced by closed-vocabulary prompting.

Adherence to this vocabulary is enforced at the generation level via closed-vocabulary prompting, so that the output is always a valid ontology term regardless of signal quality. The LLM additionally returns a confidence rating (high, medium, or low) and a brief reasoning string. Low-confidence assignments are flagged automatically for human review and are not committed to the TBox until inspected.

As illustrated in Figure 4, the signals provide quantitative, corpus-grounded evidence; the LLM makes the final decision, using domain knowledge to resolve cases where signals conflict, are absent, or point in the wrong direction. The last case is a known extraction artifact: the graph extraction model does not always direct edges consistently, so S1 and S2 may nominate swapped domain and range values. The LLM is explicitly instructed to treat signals as hints and defer to insurance semantics, allowing it to detect and correct these inversions.

Stage 5: Hierarchy Construction

Stage 5 organizes the class clusters produced in Stage 3 into a formal `rdfs:subClassOf` hierarchy. Because Stage 3 classes are extracted bottom-up from corpus evidence, they lack organizing abstractions at the top. This stage addresses this via two phases.

Phase 1: Abstract Class Discovery: A dedicated LLM call examines the full inventory of Stage 3 class names and definitions and proposes a small set of abstract top-level classes that are domain-appropriate but do not appear in the corpus directly. This is necessary because bottom-up extraction surfaces only the concepts explicitly named in documents. A corpus of insurance contracts may yield `SuicideExclusion` and `WarExclusion` as distinct classes, each grounded in specific policy language, while the organizing abstraction `Exclusion` was never named as such in any document. The abstract class discovery step recovers these missing abstractions. Proposed classes are constrained to be mutually exclusive in intent, preventing competing proposals from being nominated for the same children.

Phase 2: Per-class Placement: Each class is placed using four signals.

(i) Graph-native parent resolved in Stage 3 is the strongest signal: a role consistently observed under a given entity type directly encodes a `subClassOf` hypothesis. For example, the role *policy owner* observed under entity type PERSON strongly indicates `PolicyOwner` ⊑ `Person`.

(ii) Source-profile similarity is computed from the full empirical distribution of source entity types stored per edge label in the Stage 2 `meta` field. For each class, a profile vector is constructed over edge labels weighted by frequency as source type. Two classes with high cosine similarity over this distribution are subsumption candidates: if `PolicyOwner` and `InsuredPerson` both appear as frequent source types of the same set of edge labels, their structural positions in the corpus are similar, indicating a likely subsumption or close common-parent relationship.

(iii) Range-profile similarity is constructed symmetrically to (ii): a profile vector is built over edge labels weighted by frequency as target type, and two classes with high cosine similarity over this distribution are sibling candidates under a common parent, as they occupy equivalent structural positions on the receiving end of the same relations.

(iv) Property domain overlap is derived independently from Stage 4: two classes each assigned as the domain of the same object properties share the same relational commitments and are therefore sibling or subsumption candidates. Unlike signal (i), which arises from entity role co-occurrence in the graph, this signal is grounded in the LLM domain assignments from Stage 4 and can surface relationships between classes that never co-occurred as roles in the corpus.

As shown in Figure 5, all four signals, annotated with their strength, are presented to the LLM in a structured prompt. The LLM selects one or more parents from a closed vocabulary of corpus-derived classes, abstract classes, and `owl:Thing`, enforced via guided decoding, and returns a confidence rating and brief reasoning string per assignment. Low-confidence assignments are flagged automatically for human review.

Cycle resolution: Because placement calls are independent, the resulting hierarchy representation may contain directed cycles, which are invalid in OWL. After all placements are collected, the hierarchy is checked iteratively via depth-first search. To handle discovered two-node cyclic relationships such as `Amount` is a `subClassOf` `MonetaryValue` and `MonetaryValue` is a `subClassOf` `Amount`, a dedicated prompt presents the definitions of both classes to the LLM and asks which subsumption direction is semantically more correct. The losing direction is removed and the winner is retained. For a cycle of length more than 2, the minimal cycle path is reconstructed via breadth-first search from the back-edge endpoints. The full path, class definitions, current parents, and a set of valid replacement anchors are presented to the LLM, which identifies the semantically weakest edge in the cycle and, if removing that edge makes a class parentless, proposes a replacement parent from the set. If the LLM call fails or returns an invalid response, a confidence-based fallback removes the edge involving the lowest-confidence placement and elevates the affected class to its nearest valid ancestor. Passes repeat until no cycles remain.

Stage 6: Restriction Induction

This stage converts the restriction-stream clusters from stages 3 and 4 into a final set of OWL TBox axiom patterns. Each cluster carries a `restriction_family` label given by Stage 3, encoding the logical structure of the constraint (e.g., `cardinality_constraint`, `disjointness`, `eligibility_condition`), along with typed domain and range assignments from Stage 4.

For each cluster, a candidate is assembled by collecting the same complementary evidence sources from Stage 4 (except S2): the edge-topology distribution over entity types, the raw sentences from fact nodes whose names match the cluster, and the thematic chunk topics from those same facts. These are presented jointly to the LLM so it can cross-check its decisions against corpus-grounded evidence at multiple levels of abstraction: structural, sentential, and thematic. The allowed OWL construct set is derived deterministically from `restriction_family`, so the LLM selects from a small pre-constrained set rather than choosing freely among all OWL operators. Property names are resolved by token-overlap matching against the closed property vocabulary, since Stage 3 assigns clause-level names to restriction clusters (e.g., `PolicyLapseAfterGracePeriod`) rather than relational predicates. When Stage 4's range is uninformative, the filler is recovered by scanning the cluster definition for known class names.

The LLM returns a confidence rating alongside each formalization. Low-confidence results are separated, preserving the evidence for human inspection. A final deterministic coherence pass deduplicates identical axiom strings, and resolves contradicting cardinality assertions on the same property.

Stage 7: TBox Compilation

This final stage assembles the outputs of all preceding stages into a single, formally valid OWL TBox serialized in Turtle [9] and RDF/XML. The assembly draws from four sources: Stage 5 for the class hierarchy, Stage 3 to back-fill missing definitions and surface variants, Stage 4 for annotated property clusters, and Stage 6 for the induced restriction axioms. Because all ontological decisions, including naming, domain and range assignment, hierarchical placement, and restriction formalization, have already been made and recorded by the preceding stages, this assembly is entirely deterministic: no LLM call is made and no new decision is introduced. Terms produced with low confidence are included but annotated as such, so that a human auditor can identify and inspect uncertain terms directly within the TBox without consulting any intermediate pipeline file.

Traceability: A central property of the assembled TBox is end-to-end provenance. Because each stage preserves its decision artifacts, including the graph-native parent signal and placement confidence from Stage 5, the domain and range confidence and reasoning string from Stage 4, the surface variant cluster members and their corpus frequencies from Stage 3, and the evidence sentences and axiom family from Stage 6, the assembly has access to a rich chain of evidence for every class, property, and restriction it emits. This chain is materialized as human-readable annotations on each emitted term, making the full decision path inspectable without consulting any intermediate file. For example, `PolicyOwner`'s provenance record shows five surface variants, including *policy owner*, *issuing policy owner*, and *owner of the policy*, merged by Stage 3, its assignment as domain of `hasPremiumPaymentFrequency` with high confidence from Stage 4, and its placement as a subclass of `Person` via a strong graph-native parent signal from Stage 5 — tracing a continuous path from raw policy text to the final OWL declaration. Since ontology induction is inherently uncertain, this provenance makes errors diagnosable and trust in the output evidence-grounded rather than assumed.

ABox Population

Given the assembled TBox and the per-document typed pools produced by Stage 1, we also design a method for producing a populated OWL ABox for each contract, asserting one named individual per entity and grounding it in the class and property vocabulary established by the TBox. ABox construction proceeds in three steps.

Individual Minting: For each entity record, we resolve a set of OWL class names by looking up its type (e.g., `PERSON`) and each of its atomic roles (e.g., *policy owner*) in indexes built from Stages 3 and 4. When multiple classes are resolved, we select the most derived one in the TBox hierarchy as the canonical class for that individual. Two entity nodes are merged into a single individual if and only if they share the same canonical class and the same surface name, preventing entities with the same name but different roles from being incorrectly collapsed. Each individual is assigned an Internationalized Resource Identifier (IRI) built from its name and contract identifier, and receives `rdf:type` triples for all resolved classes, a name literal, and any key-value attributes attached to its Stage 1 entity record (e.g., `dateOfBirth`, `faceAmount`).

Object Property Assertion: We assert object property triples from the edge buckets produced by Stage 1. For `entity_entity` edges, we look up the edge label in the property index from Stages 3 and 4 and assert the triple directly between the two individuals. For fact-mediated edges, we first identify the primary subject of each fact from the `entity_fact` bucket, then pair it with the target entity from the `fact_entity` bucket to form the triple. Edges whose label does not resolve to a known property, or whose source individual's type does not match the property's assigned domain, are discarded.

Datatype Property Mining: Fact sentences often contain literal values such as amounts, dates, and durations that cannot be read directly from graph structure. For each fact with a resolvable subject individual, we issue a bounded LLM call with the fact sentence and a closed vocabulary of `owl:DatatypeProperty` terms drawn from Stages 3 and 4. The model returns property-value pairs, each of which is validated against the vocabulary before being asserted as a literal triple. This follows the same design principle as the TBox pipeline: the graph defines the structure and constrains the vocabulary, while the LLM fills in bounded semantic judgments within those constraints.

4 Evaluation

We evaluate our pipeline in the life insurance domain using three complementary evaluation setups, each targeting a different aspect of ontology quality and functional usability. Our evaluation is confined to the life insurance domain; however, since every pipeline stage operates on domain-agnostic graph signals rather than insurance-specific rules or lexicons, we expect the architecture

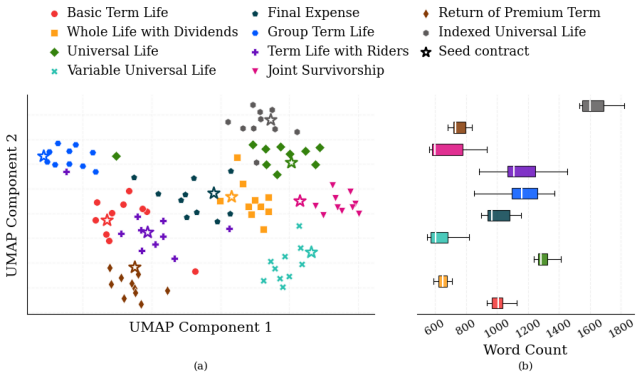


Figure 6: (a) **UMAP [32] projection of the 100-contract corpus.** Contracts naturally separate by product type, yet within each cluster the contracts show substantial within-cluster dispersion, confirming that variants are diverse rather than near-duplicates of the seed contracts. (b) **Word-count distribution per product type.**

to transfer to other high-stakes, clause-intensive domains, leaving empirical validation to future work.

4.1 Datasets

Talukder et al. [46] study automated ontology generation from life insurance contracts, comparing a direct single-agent LLM baseline against a multi-agent architecture. Their work provides two publicly available synthetic term life insurance contracts: Equivita and Sentinel, along with automatically derived CQs, and evaluates generated ontologies on architectural quality scored by a panel of LLM judges across Extensibility, Redundancy, and Ontology Design Pattern (ODP) Usage, and functional CQ coverage (via SPARQL and RAG-based evaluation). We adopt their contracts, CQ sets, and evaluation setup directly to compare against their baselines under identical conditions.

Mridul et al. [33] provide a benchmark for gap and overlap analysis as a test of knowledge graph task readiness. It consists of ten expert-verified synthetic life insurance contracts spanning the full range of major product categories, a manually constructed ground-truth domain ontology (TBox) with a fully populated and evidence-annotated ABox where every individual is anchored to a contract identifier, section reference, and verbatim clause text, and 58 structured scenarios each paired with SPARQL queries, contract-level ground-truth outcome labels (COVERED, DENIED, NOT_APPLICABLE), and clause-level justifications. The scenarios were generated using an LLM-assisted workflow and subsequently curated by a domain expert, with eight expert-authored edge cases probing complex legal nuances. The ground-truth ontology and its SPARQL queries achieve 100% accuracy by construction, and the task complexity is confirmed by the fact that state-of-the-art LLMs reading the contracts directly achieve only 65–87% accuracy. We use this benchmark as a primary evaluation substrate and extend it from ten to one hundred contracts for scalability analysis.

Extended 100-contract dataset. We generated an extended 100-contract synthetic corpus using Claude Sonnet 4.6 [4], Gemini-3 [21], and ChatGPT-5.3 [39], using the original ten contracts as product-type seeds. Figure 6(a) shows a UMAP [32] projection of the resulting corpus. The contracts separate clearly by product type, and the embedding visualization provides no obvious evidence of collapse to near-duplicates of the seed documents. Within each cluster, the generated variants spread across the embedding space, indicating meaningful structural diversity. Word-count distributions per product type are shown in Figure 6(b).

Table 1: CQ Coverage Metric comparing our pipeline against the direct generation and multi-agent baselines from Talukder et al. [46].

Sample	Method	Direct	Multi-Agent	Ours
Equivita	SPARQL	0.6304	0.6196	**0.8527**
	RAG	0.5994	**0.7112**	0.6646
Sentinel	SPARQL	0.4027	0.4383	**0.7694**
	RAG	0.4450	0.5510	**0.6812**

Table 2: Architectural quality metrics comparing our pipeline against the direct generation and multi-agent baselines from Talukder et al. [46]. Scores are averaged across 5 LLM judges. Inter-judge reliability measured using **Krippendorff's** α [23] over the combined judged artifact set is **0.71** for Extensibility, **0.57** for Redundancy, **0.61** for ODP Usage, and **0.73** overall.

Dataset	Metric	Direct	Multi-Agent	Ours
Equivita	Extensibility	1.60	3.80	3.60
	Redundancy	1.00	2.40	**2.60**
	ODP Usage	1.40	**3.80**	3.60
	Overall	1.33	**3.33**	3.27
Sentinel	Extensibility	2.80	3.80	**4.00**
	Redundancy	2.00	1.80	**3.10**
	ODP Usage	2.20	3.40	**3.50**
	Overall	2.33	3.00	**3.53**

4.2 Architectural Quality and CQ Coverage

Tables 1 and 2 compare our pipeline against the direct generation and multi-agent baselines from Talukder et al. [46] on their Equivita and Sentinel contracts.

On SPARQL-based CQ coverage (Table 1), our pipeline outperforms both baselines. The RAG-based scores follow the same trend on Sentinel, where we achieve 0.6812 against 0.4450 and 0.5510. On Equivita RAG, our score of 0.6646 is below the multi-agent score of 0.7112. RAG evaluation synthesizes retrieved ontology subgraphs into natural language answers before adjudication, which can favor ontologies whose graph topology and labeling patterns align well with the embedding-based retrieval used in that specific evaluation setup. This is a known sensitivity of RAG-based metrics to surface representation choices, and the result does not reflect a gap in knowledge coverage, as our strong SPARQL score on the same contract confirms.

On architectural quality (Table 2), our pipeline performs on par with or better than the multi-agent baseline and substantially above the direct generation baseline on both contracts. The redundancy improvement is meaningful because it is architecturally the hardest dimension to optimize: it requires global semantic consistency across the ontology, which LLM-based systems struggle with due to attention limitations and the absence of an external concept registry. Our embedding-based clustering stage directly addresses this

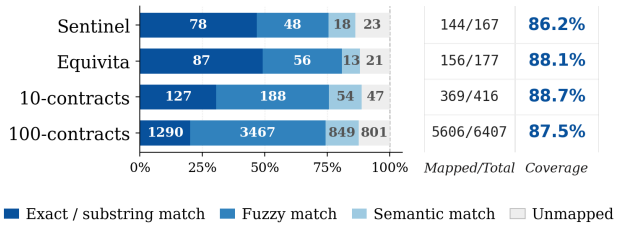


Figure 7: Ontology keyphrase coverage analysis across contract sets — Sentinel and Equivita from Talukder et al. [46], 10-contract dataset from Mridul et al. [33], and our extended 100-contract dataset. For each dataset, keyphrases extracted from the contracts are matched against TBox artifacts using **substring, fuzzy, and semantic similarity matching.**

by deduplicating surface variants before any OWL term is introduced. We note that these scores are produced by LLM judge panels on a coarse 1–5 scale using relatively generic evaluation prompts, so narrow score differences should not be over-interpreted. What the results establish clearly is that our pipeline does not sacrifice structural quality for coverage gains.

4.3 Ontology Keyphrase Coverage

We apply the keyphrase coverage methodology from Mridul et al. [33], which extracts domain keyphrases from contracts using Claude Sonnet 4.6 and matches them against TBox artifacts using three complementary methods: exact/substring matching, fuzzy matching (RapidFuzz[2] token-sort ratio ≥ 70), and semantic similarity (SentenceTransformer[3] cosine similarity ≥ 0.70). A keyphrase is considered covered if any method produces a match.

Figure 7 reports results across all four evaluation corpora. Our pipeline achieves strong coverage in all settings: 86.2% for Sentinel, 88.1% for Equivita, 88.7% for the 10-contract dataset, and 87.5% for the 100-contract dataset. These figures are consistent with the 84.6% coverage reported by Mridul et al. [33] for their hand-constructed ontology. As the original benchmark paper notes, the small unmapped fraction does not necessarily always indicate ontology gaps. Manual inspection shows that in most cases the relevant concept is represented under a different label or through a richer structure that string-level matching cannot resolve. This pattern holds for our pipeline as well.

4.4 CQ Coverage on the 10-Contract Dataset

To evaluate functional usability against the Mridul et al. [33] benchmark, we generated 300 competency question–answer pairs (30 per contract) from the 10-contract dataset using Claude Sonnet 4.6. For each question, we generated a corresponding SPARQL query grounded in our induced ontology's vocabulary and executed it against the ABox produced by our pipeline's ABox construction stage. Query results were evaluated against the reference answers using Qwen3-30B-Instruct [48] as an LLM judge. The evaluation pipeline is illustrated in Figure 8(a).

Figure 8(b) shows per-contract accuracy scores. Our pipeline achieves 91% overall accuracy, with per-contract scores ranging

[2]https://pypi.org/project/RapidFuzz
[3]https://pypi.org/project/sentence-transformers

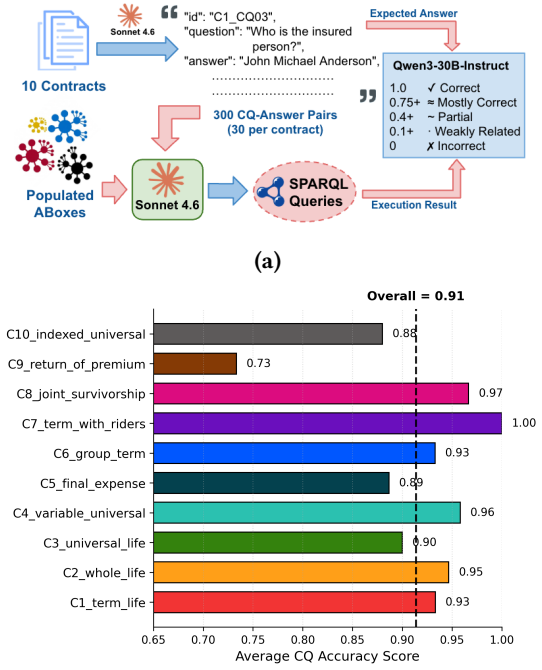


Figure 8: CQ evaluation on the 10-contract dataset [33]. (a) Pipeline: A total of 300 CQs are generated using Claude Sonnet 4.6, along with SPARQL queries grounded in the populated ABoxes. Query execution results are compared against ground truth answers using Qwen3-30B-Instruct as an LLM judge. **(b) CQ Coverage Scores:** The generated ontology can answer CQs with high accuracy (91% overall accuracy).

from 0.73 to 1.00. Strong performance holds even for structurally complex products such as C4 (Variable Universal Life, 0.96) and C10 (Indexed Universal Life, 0.88), demonstrating that the induced ontology and populated ABox are sufficient to answer the large majority of factual CQs derivable from the source contracts.

4.5 Gap and Overlap Analysis

We evaluate our generated ontology against the gap and overlap benchmark of Mridul et al. [33]. Given a scenario describing an insurance event with specific conditions, the task is to determine for each contract whether the scenario is Covered, Denied, or Not_Applicable, with justification traceable to contract clauses. The 58 benchmark scenarios probe genuine structural differences across contracts and are confirmed to be non-trivial: reading contract text directly, Claude Sonnet 4.6 achieves 87.76%, ChatGPT-5.3 achieves 72.93%, and Gemini-3 achieves 65.17%, despite the strong document-level question-answering capabilities of state-of-the-art LLMs.

The benchmark provides ground-truth SPARQL queries written against the ground truth ontology. To evaluate our induced ontology, we rewrote these queries using our ontology's vocabulary via ChatGPT-5.5 [40]. Table 3 reports the results. Our induced ontology achieves 86.38% accuracy, essentially matching the best-performing LLM baseline (Claude at 87.76%) and outperforming the others. This is a strong result: an automatically induced ontology approaches the

Table 3: Evaluation on the gap and overlap analysis benchmark [33]. The generated ontology outperforms ChatGPT-5.3 and Gemini-3 and performs within 1.38 percentage points of Claude Sonnet 4.6, demonstrating strong task-level readiness.

Method	Accuracy
LLM Based - Claude Sonnet 4.6	87.76% (509/580)
LLM Based - ChatGPT-5.3	72.93% (423/580)
LLM Based - Gemini-3	65.17% (378/580)
Ontology Based - Ground Truth	100.00% (580/580)
Ontology Based - Ours	**86.38%** (501/580)

strongest direct-LLM baseline without manual TBox engineering, while providing deterministic reproducibility and explicit provenance that LLM inference often cannot offer. The remaining errors reflect modeling imprecisions that can be localized through the pipeline's provenance annotations.

4.6 Ontology Growth and Scalability

Figure 9 tracks cumulative ontology elements as a function of documents ingested for both the 10-contract and 100-contract datasets. Both datasets exhibit the same pattern: rapid initial growth as the pipeline encounters new concepts and relations, followed by a steadily decelerating rate of acquisition as the vocabulary saturates. This is the expected behavior of a well-functioning induction pipeline and confirms that the ontology is converging toward a stable domain vocabulary rather than growing unboundedly. In practice, once an ontology has been induced from a sufficiently large corpus, new documents in the same domain will require only incremental updates. In our pipeline, new documents can be appended to the corpus and the pipeline re-run, with the clustering and deduplication stages naturally absorbing known variants and emitting genuinely novel terms where they exist.

4.7 Auditable Provenance

Beyond quantitative metrics, one of the most practically significant properties of our pipeline is its comprehensive provenance architecture. Every ontology term carries a complete decision chain: surface variant cluster members and their corpus frequencies from Stage 3, domain and range confidence scores and LLM reasoning strings from Stage 4, graph-native parent signals and placement confidence from Stage 5, and evidence sentences and axiom families from Stage 6. This chain is materialized as human-readable annotations directly within the TBox, allowing any term to be traced back through every pipeline decision to the original document passages that grounded it.

This transforms the induced ontology from a black-box output into an auditable artifact. Even where the generated ontology is not perfect, every uncertain decision is flagged automatically and its supporting evidence is preserved. A human reviewer can inspect these flagged terms, examine the graph evidence and LLM reasoning behind each decision, and apply targeted corrections with full context. Rather than constructing a domain ontology from scratch, a practitioner can run our pipeline, obtain a high-quality draft with the large majority of terms correct, and focus human effort on reviewing and refining the small uncertain subset. This

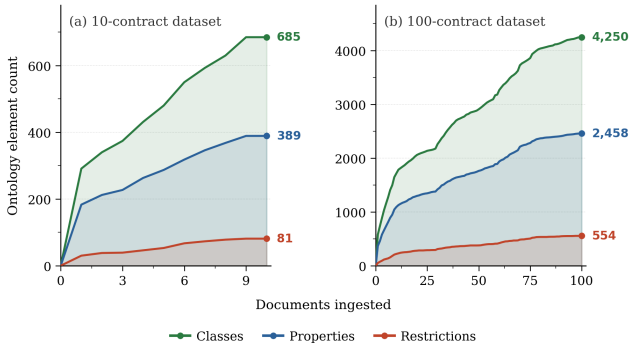


Figure 9: Ontology growth curves for the 10-contract and 100-contract datasets. Each curve tracks cumulative ontology elements against documents ingested, showing rapid initial growth followed by decelerating acquisition as the vocabulary saturates.

represents a fundamentally more scalable and tractable approach to ontology engineering.

5 Conclusion

We presented a graph-grounded pipeline for ontology induction from domain documents that treats LLMs as bounded semantic oracles rather than free generators. By first converting documents into UDH and restricting every LLM call to closed-vocabulary prompting over corpus-derived evidence, the pipeline produces a complete, provenance-annotated OWL TBox and populated ABox without any unconstrained generation step. Evaluated on the life insurance domain, the induced ontology outperforms direct and multi-agent LLM baselines on CQ coverage, matches the best-performing LLM on structured gap-and-overlap reasoning without manual TBox engineering, and attains keyphrase coverage comparable to a hand-constructed reference ontology. Ontology growth analysis confirms that the pipeline converges toward a stable domain vocabulary rather than growing unboundedly, supporting practical deployment at scale.

Several directions remain open for future work. Because the pipeline's design imposes no insurance-specific structure, its architecture generalizes in principle to other high-stakes document-heavy domains such as legal contracts, clinical guidelines, or financial regulation; empirically validating this transfer is a natural next step. When multiple ontologies are induced independently across organizations, automatically aligning and merging them into a coherent shared vocabulary is a further open challenge that the pipeline's provenance architecture is well-positioned to support. Finally, evaluation of automatically induced ontologies remains a fundamentally hard problem; developing a unified, domain-agnostic benchmark that jointly assesses structural quality, functional coverage, and provenance fidelity would be a valuable contribution to the ontology learning community.

Acknowledgments

This research was supported by NSF IUCRC CRAFT Grant #22022. The views expressed do not necessarily reflect those of NSF IUCRC CRAFT. We thank the CRAFT Industry Board members for their advice and resources.

GenAI Usage Disclosure

During the preparation of this work, the authors used Claude Sonnet, ChatGPT, and Gemini to polish and improve the clarity of the text originally drafted by the authors, and for assistance with code segments during implementation. After using these services, the authors reviewed and edited all AI-assisted content as needed and take full responsibility for the publication's content.

References

[1] Mohammad Sadeq Abolhasani and Rong Pan. 2024. Leveraging LLM for Automated Ontology Extraction and Knowledge Graph Generation. *CoRR* (2024).

[2] Hamid Ahaggach, Lylia Abrouk, and Eric Lebon. 2023. Information extraction and ontology population using car insurance reports. In *International Conference on Information Technology-New Generations*. Springer, 405–411.

[3] Muhammad Amith, Marcelline R Harris, Cooper Stansbury, Kathleen Ford, Frank J Manion, and Cui Tao. 2022. Expressing and executing informed consent permissions using SWRL: the all of us use case. In *AMIA Annual Symposium Proceedings*, Vol. 2021. 197.

[4] Anthropic. 2026. Claude Sonnet 4.6. https://claude.ai/

[5] Webert Araújo, Gercina Lima, and Ivo Pierozzi Jr. 2016. Data-Driven Ontology Evaluation Based on Competency Questions: A Study in the Agricultural Domain. In *Knowledge Organization for a Sustainable World: Challenges and Perspectives for Cultural, Scientific, and Technological Sharing in a Connected Society*. Ergon-Verlag, 326–332.

[6] Muhammad Nabeel Asim, Muhammad Wasim, Muhammad Usman Ghani Khan, Waqar Mahmood, and Hafiza Mahnoor Abbasi. 2018. A survey of ontology learning techniques and applications. *Database* 2018 (2018), bay101.

[7] Hamed Babaei Giglou, Jennifer D'Souza, and S¨oren Auer. 2023. LLMs4OL: Large language models for ontology learning. In *International Semantic Web Conference*. Springer, 408–427.

[8] Roos M Bakker, Daan L Di Scala, and MH de Boer. 2024. Ontology learning from text: an analysis on llm performance. In *Proceedings of the 3rd NLP4KGC International Workshop on Natural Language Processing for Knowledge Graph Creation, colocated with Semantics*. 17–19.

[9] David Beckett, Tim Berners-Lee, Eric Prud'hommeaux, and Gavin Carothers. 2014. RDF 1.1 turtle: Terse RDF triple language. *W3C recommendation* 25 (2014).

[10] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2024. Guiding llms the right way: Fast, non-invasive constrained generation. *arXiv preprint arXiv:2403.06988* (2024).

[11] Camila Bezerra, Fred Freitas, and Filipe Santana. 2013. Evaluating ontologies with competency questions. In *2013 IEEE/WIC/ACM International Joint Conferences on Web Intelligence (WI) and Intelligent Agent Technologies (IAT)*, Vol. 3. IEEE, 284–285.

[12] Haonan Bian. 2025. LLM-empowered knowledge graph construction: A survey. *arXiv preprint arXiv:2510.20345* (2025).

[13] Chris Biemann. 2005. Ontology learning from text: A survey of methods. *Journal for Language Technology and Computational Linguistics* 20, 2 (2005), 75–93.

[14] Markos Charalambous, Aristeidis Farao, George Kalantzantonakis, Panagiotis Kanakakis, Nikos Salamanos, Evangelos Kotsifakos, and Evangellos Froudakis. 2022. Analyzing coverages of cyber insurance policies using ontology. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*. 1–7.

[15] Rafi Rashid Chowdhury, Takaaki Goto, Kensei Tsuchida, Tadaaki Kirishima, and Ajay Bandi. 2025. An Automated Framework of Ontology Generation for Abstract Concepts Using LLMs. In *International Conference on Computers and Their Applications*. Springer, 170–180.

[16] Philipp Cimiano. 2006. *Ontology learning and population from text: algorithms, evaluation and applications*. Springer.

[17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.

[18] Rick Du, Huilong An, Keyu Wang, and Weidong Liu. 2024. A short review for ontology learning: Stride to large language models trend. *arXiv preprint arXiv:2404.14991* (2024).

[19] Mirna El Ghosh, Hala Naja, Habib Abdulrab, and Mohamad Khalil. 2017. Towards a legal rule-based system grounded on the integration of criminal domain ontology and rules. *Procedia computer science* 112 (2017), 632–642.

[20] Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. 2025. Generating structured outputs from language models: Benchmark and studies. *arXiv e-prints* (2025), arXiv–2501.

[21] Google DeepMind. 2025. Gemini 3. https://gemini.google.com/

[22] Thomas R Gruber. 1993. A translation approach to portable ontology specifications. *Knowledge acquisition* 5, 2 (1993), 199–220.

[23] Andrew F Hayes and Klaus Krippendorff. 2007. Answering the call for a standard reliability measure for coding data. *Communication methods and measures* 1, 1 (2007), 77–89.

[24] Marti A Hearst. 1992. Automatic acquisition of hyponyms from large text corpora. In *COLING 1992 volume 2: The 14th international conference on computational linguistics*.

[25] Ian Horrocks, Peter F Patel-Schneider, Harold Boley, Said Tabet, Benjamin Grosof, Mike Dean, et al. 2004. SWRL: A semantic web rule language combining OWL and RuleML. *W3C Member submission* 21, 79 (2004), 1–31.

[26] Vamsi Krishna Kommineni, Birgitta K¨onig-Ries, and Sheeba Samuel. 2024. From human experts to machines: An LLM supported approach to ontology and knowledge graph construction. *CoRR* (2024).

[27] Anna Sofia Lippolis, Mohammad Javad Saeedizade, Robin Keskis¨arkk¨a, Sara Zuppiroli, Miguel Ceriani, Aldo Gangemi, Eva Blomqvist, and Andrea Giovanni Nuzzolese. 2025. Ontology generation using large language models. In *European Semantic Web Conference*. Springer, 321–341.

[28] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-eval: NLG evaluation using gpt-4 with better human alignment. *arXiv preprint arXiv:2303.16634* (2023).

[29] Andy Lo, Albert Q Jiang, Wenda Li, and Mateja Jamnik. 2024. End-to-end ontology learning with large language models. *Advances in Neural Information Processing Systems* 37 (2024), 87184–87225.

[30] Ravi Lourdusamy and Stanislaus Abraham. 2019. A survey on methods of ontology learning from text. In *International Conference on Information, Communication and Computing Technology*. Springer, 113–123.

[31] Alexander Maedche and Steffen Staab. 2001. Ontology learning for the semantic web. *IEEE Intelligent systems* 16, 2 (2001), 72–79.

[32] Leland McInnes, John Healy, and James Melville. 2018. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426* (2018).

[33] Maruf Ahmed Mridul, Rohit Kapa, and Oshani Seneviratne. 2026. A Benchmark for Gap and Overlap Analysis as a Test of KG Task Readiness. *arXiv preprint arXiv:2604.10853* (2026).

[34] Daniel Müllner. 2011. Modern hierarchical, agglomerative clustering algorithms. *arXiv preprint arXiv:1109.2378* (2011).

[35] Muhammad Raza Naqvi[1], Syed Khuram Shahzad, Muhammad Waseem Iqbal, and Maryam Al-Thawadi. 2023. Ontology-Driven Smart Health Insurance. In *Soft Computing Applications: Proceedings of the 9th International Workshop Soft Computing Applications (SOFA 2020)*, Vol. 1438. Springer Nature, 51.

[36] Roberto Navigli and Paola Velardi. 2004. Learning domain ontologies from document warehouses and dedicated web sites. *Computational Linguistics* 30, 2 (2004), 151–179.

[37] Sanaz Saki Norouzi, Adrita Barua, Antrea Christou, Nikita Gautam, Andrew Eells, Pascal Hitzler, and Cogan Shimizu. 2025. Ontology population using LLMs. In *Handbook on Neurosymbolic AI and Knowledge Graphs*. IOS Press, 421–438.

[38] Natalya F Noy, Deborah L McGuinness, et al. 2001. Ontology development 101: A guide to creating your first ontology.

[39] OpenAI. 2026. ChatGPT (Version 5.3). https://chatgpt.com/

[40] OpenAI. 2026. ChatGPT (Version 5.5). https://chatgpt.com/

[41] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering* 36, 7 (2024), 3580–3599.

[42] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 3982–3992.

[43] Stephen Roller, Douwe Kiela, and Maximilian Nickel. 2018. Hearst patterns revisited: Automatic hypernym detection from large text corpora. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 358–363.

[44] Mohammad Javad Saeedizade and Eva Blomqvist. 2024. Navigating ontology development with large language models. In *European Semantic Web Conference*. Springer, 143–161.

[45] Rudi Studer, V Richard Benjamins, and Dieter Fensel. 1998. Knowledge engineering: Principles and methods. *Data & knowledge engineering* 25, 1-2 (1998), 161–197.

[46] Abid Talukder, Maruf Ahmed Mridul, and Oshani Seneviratne. 2026. Towards Automated Ontology Generation from Unstructured Text: A Multi-Agent LLM Approach. *arXiv preprint arXiv:2604.23090* (2026).

[47] Abid Talukder, Maruf Ahmed Mridul, and Oshani Seneviratne. 2026. Unified Discourse Hypergraph: Rich graph representation combining discourse graph and hypergraph. https://github.com/brains-group/unified_discourse_hyper_graph.

[48] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[49] Tania Tudorache. 2020. Ontology engineering: Current state, challenges, and future directions. *Semantic Web* 11, 1 (2020), 125–138.

[50] Paola Velardi, Stefano Faralli, and Roberto Navigli. 2013. Ontolearn reloaded: A graph-based algorithm for taxonomy induction. *Computational Linguistics* 39, 3 (2013), 665–707.

[51] Brandon T Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702* (2023).

[52] Yuan Yao, Deming Ye, Peng Li, Xu Han, Yankai Lin, Zhenghao Liu, Zhiyuan Liu, Lixin Huang, Jie Zhou, and Maosong Sun. 2019. DocRED: A large-scale document-level relation extraction dataset. In *Proceedings of the 57th annual meeting of the association for computational linguistics*. 764–777.

[53] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.

[54] Yuqi Zhu, Xiaohan Wang, Jing Chen, Shuofei Qiao, Yixin Ou, Yunzhi Yao, Shumin Deng, Huajun Chen, and Ningyu Zhang. 2024. Llms for knowledge graph construction and reasoning: Recent capabilities and future opportunities. *World Wide Web* 27, 5 (2024), 58.